

**INSTALASI PROYEK LARAVEL & ROUTING
DASAR**

INSTALASI PROYEK LARAVEL & ROUTING DASAR

Menguasai Struktur Proyek, HTTP Verbs, Parameter Routing, dan Pola CRUD Modern

Pemrograman Berbasis Kerangka Kerja (PBKK) — Pertemuan 2

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas (FTEIC)

Institut Teknologi Sepuluh Nopember (ITS) Surabaya

Menjembatani Pertemuan 1 & 2: Dari Teori Ke Praktik

Review Pertemuan 1

- **Evolusi PHP Native:** Dari arsitektur Spaghetti Code terintegrasi tanpa pengorganisasian menuju pemisahan terstruktur.
- **Prinsip DRY:** Don't Repeat Yourself – menolak duplikasi kode include secara beruntun dan rawan eror.
- **Model-View-Controller:** Pemisahan model representasi data, view penyaji, dan controller pengatur logika.
- **Ekosistem Modern:** Mengenali keunggulan framework Laravel di kancah industri global masa kini.

Agenda Utama Pertemuan 2

- **Instalasi Proyek:** Memasang dependensi Composer dan inisialisasi framework Laravel murni.
- **Anatomi Direktori:** Membedah struktur folder app, routes, resources, database, config, dan file konfigurasi .env.
- **Dasar Routing:** Mempelajari URL-to-Action, HTTP verb GET & POST, serta Artisan route:list.
- **Parameter Routing:** Menerapkan parameter wajib (required), parameter opsional, dan named routing.

BAGIAN 01

INSTALASI PROYEK & ANATOMI DIREKTORI LARAVEL

Memulai Perjalanan: Inisialisasi Proyek Baru via Composer

Composer: Pengelola Dependensi

- **Manajer Paket PHP:** Composer bertindak sebagai package manager resmi untuk menjaga ketersediaan pustaka eksternal.
- **Perintah Inisialisasi:** Menggunakan `create-project` untuk meluncurkan kerangka proyek bersih langsung dari package store resmi.
- **Struktur Proyek:** Menyusun skema dependensi lengkap yang siap dikontrol lewat Artisan CLI.

// Langkah Instalasi lewat Terminal

```
# Metode 1: Menggunakan Composer create-project secara langsung  
composer create-project laravel/laravel toko-online
```

```
# Masuk ke folder proyek  
cd toko-online
```

```
# Menjalankan Local Web Server  
php artisan serve
```

```
# Metode 2: Menggunakan Laravel Installer global  
composer global require laravel/installer  
laravel new toko-online-2
```

Artisan CLI: Asisten Baris Perintah Terintegrasi Anda

Konsep Artisan CLI

- **Tool Baris Perintah:** Artisan adalah antarmuka baris perintah (command line interface) bawaan Laravel yang sangat kuat.
- **Otomatisasi Boilerplate:** Membantu pengembang mempercepat pembuatan kelas controller, model, migration, seeder, dsb.
- **Manajemen Server:** Menyediakan built-in local development server tanpa perlu menyiapkan server Apache/Nginx eksternal.

Perintah-Perintah Artisan Dasar

- **php artisan serve:** Menjalankan aplikasi di server lokal (<http://127.0.0.1:8000>) untuk pengujian cepat.
- **php artisan help:** Menampilkan panduan detail penggunaan dan dokumentasi setiap command.
- **php artisan --version:** Kecepatan tinggi melihat versi framework Laravel yang aktif dalam proyek.

Anatomi Struktur Folder: Fokus Pada app/ dan bootstrap/

Folder app/ — Jantung Aplikasi

- **Pusat Logika Bisnis:** Folder paling krusial tempat hampir semua kode logika back-end ditulis.
- **Sub-direktori Http/:** Menyimpan Controllers, Middleware, dan Requests pembentuk respon sistem.
- **Sub-direktori Models/:** Menyimpan kelas representasi database yang terhubung langsung dengan Eloquent ORM.

Folder bootstrap/ — Mesin Penggerak

- **Proses Bootstrapping:** Direktori yang mengontrol jalannya inisialisasi awal (startup runtime) dari framework Laravel.
- **File app.php:** Mengatur konfigurasi routing, penanganan exception (error), dan pendaftaran middleware global.
- **Folder cache/:** Berisi file cache internal yang dioptimalkan framework untuk mempercepat performa request web.

Anatomi Struktur Folder: Menguak config/ dan database/

Folder config/ — Sentralisasi Aturan

- **Konfigurasi Terpusat:** Berisi seluruh file konfigurasi sistem seperti database.php, app.php, session.php, dan auth.php.
- **Integrasi dengan .env:** Nilai di dalam config membaca variabel lingkungan (.env) agar lebih aman dan dinamis.
- **Fleksibilitas Sistem:** Memudahkan penyuntingan perilaku framework tanpa perlu mengubah core engine internal.

Folder database/ — Fondasi Data

- **Migrations/:** Menyimpan berkas kontrol versi tabel database secara terstruktur lewat bahasa pemrograman PHP.
- **Seeders/:** Berisi script pengisian data awal/default secara otomatis untuk kebutuhan testing.
- **Factories/:** Memanfaatkan pustaka Faker untuk memproduksi data dummy skala besar secara acak dan cepat.

Anatomi Struktur Folder: Menjelajahi `resources/` dan `routes/`

Folder `resources/` — Sisi Frontend

- **Views/:** Folder tempat file Blade (`.blade.php`) diletakkan sebagai kerangka visual halaman.
- **Aset CSS & JS:** Menyimpan file stylesheet dan JavaScript yang akan dikompilasi oleh Vite Asset Bundler.
- **Bahasa/Localization:** Menyimpan file json atau php untuk menangani translasi multi-bahasa aplikasi.

Folder `routes/` — Gerbang Arah

- **web.php:** Tempat pendaftaran seluruh rute untuk antarmuka web biasa yang dilengkapi session dan proteksi CSRF.
- **api.php:** Rute endpoint RESTful API tanpa status (stateless) yang secara default menggunakan token.
- **console.php:** Mengonfigurasi command baris perintah custom yang dapat dieksekusi via Artisan CLI.

Menguasai Berkas .env: Manajemen Konfigurasi Lingkungan

Fungsi & Keamanan .env

- **Pemisahan Lingkungan:** Memisahkan detail koneksi antara tahap Development, Testing, dan Production.
- **Keamanan Kredensial:** Menghindari hardcoding sandi database, API key, dan kunci enkripsi aplikasi langsung di dalam kode.
- **Aturan Emas:** File .env TIDAK boleh dikirim ke repository Git (otomatis terdaftar di .gitignore)!

```
// Anatomi Standar File .env
```

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:XyZ123abc456...
APP_DEBUG=true
APP_URL=http://localhost
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=toko_online
DB_USERNAME=root
DB_PASSWORD=sandi_aman
```

```
CACHE_STORE=database
QUEUE_CONNECTION=sync
```

Mengenai Autoloading PSR-4 & Peran Folder vendor/

Autoloading Standar PSR-4

- **Otomatisasi Import:** Menghilangkan keharusan menuliskan puluhan baris ``require`` atau ``include`` di awal berkas.
- **Pola Namespace:** Memetakan direktori ke representasi namespace PHP yang rapi (misalnya, ``namespace App\Http\Controllers;``).
- **Sinergi Composer:** Composer memetakan rute autoloading secara instan saat proyek di-load.

Folder vendor/ — Gudang Dependensi

- **Pustaka Eksternal:** Tempat bersarangnya framework Laravel core dan paket-paket komposer pendukung.
- **Manajemen Otomatis:** Dikelola sepenuhnya oleh Composer, dilarang mengubah isi folder ini secara manual.
- **Ukuran Besar:** Folder ini diabaikan oleh Git, hanya diunduh saat instalasi awal lewat ``composer install``.

BAGIAN 02

PENGANTAR SISTEM ROUTING DI LARAVEL

Konsep Dasar Routing: Bagaimana URL Dipetakan ke Aksi

Pengertian Rute (Route)

- **Pintu Masuk Aplikasi:** Rute bertindak sebagai pintu masuk utama yang menyaring setiap URL yang diketik pengguna.
- **Pemetaan Aksi:** Menentukan dengan tepat baris kode mana (Closure atau Controller method) yang akan memproses permintaan.
- **Definisi Terpusat:** Seluruh alur navigasi web dideklarasikan secara ringkas dalam berkas routes/web.php.

Keunggulan Dibanding PHP Native

- **URL Bersih (Pretty URL):** Menghilangkan ekstensi berkas yang mengganggu estetika (misal: /tentang vs /tentang.php).
- **Fleksibilitas Desain:** Alamat URL dapat diubah kapan saja tanpa perlu mengubah struktur berkas fisik dalam server.
- **Pemberian Nama Unik:** Dapat diberikan pengenalan unik agar tautan di halaman web bersifat dinamis.

Mengapa Menggunakan `routes/web.php` untuk Aplikasi Web?

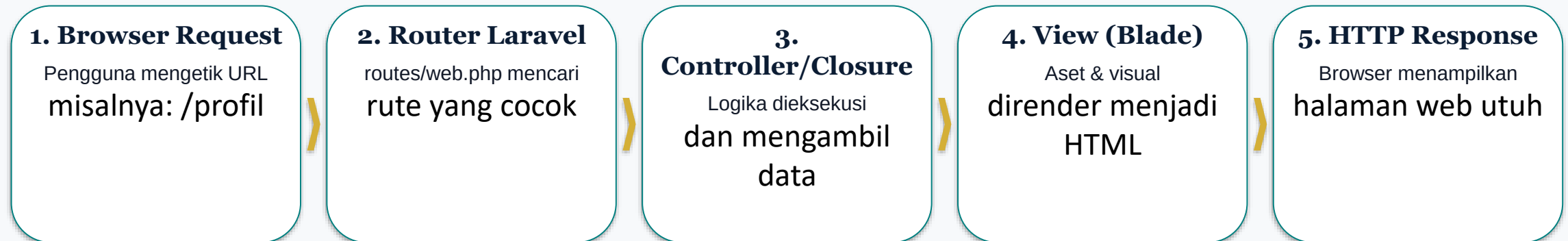
Keunikan Berkas `web.php`

- **Dilengkapi State/Session:** Rute di berkas ini secara otomatis mengaktifkan session PHP agar data pengguna tetap tersimpan.
- **Proteksi Keamanan:** Mendapatkan lapisan keamanan tambahan berupa verifikasi CSRF (Cross-Site Request Forgery) secara otomatis.
- **Cookie & Enkripsi:** Menyediakan pembacaan cookie terenkripsi yang aman untuk kebutuhan login pengguna.

Kombinasi dengan Berkas Lain

- **`routes/api.php`:** Digunakan khusus untuk rute API yang bersifat stateless (tidak menyimpan sesi) dan lebih ringan.
- **Fungsi Penyeimbang:** Pemisahan ini memastikan alur kerja web interaktif dan API eksternal berjalan mandiri tanpa interferensi.

Request Lifecycle: Alur Perjalanan Rute



HTTP Verbs: Memilih GET vs POST secara Tepat

Memahami karakteristik dan fungsionalitas utama dari dua HTTP request method terpenting pada aplikasi web.

Karakteristik	HTTP GET Request	HTTP POST Request
Tujuan Utama	Mengambil atau menampilkan data dari server.	Mengirim atau menyimpan data baru ke server.
Visibilitas URL	Parameter terlihat jelas di address bar (Query String).	Parameter tersembunyi aman di dalam Request Body.
Batas Ukuran	Sangat terbatas (maksimum sekitar 2048 karakter).	Tidak terbatas (ideal untuk upload file / form besar).
Caching & Bookmark	Dapat di-cache oleh browser dan di-bookmark.	Tidak dapat di-cache dan tidak bisa di-bookmark.
Keamanan	Kurang aman untuk kredensial sensitif (seperti sandi).	Lebih aman karena data tidak terekspos di URL.
Deklarasi Laravel	<code>Route::get('/produk', ...)</code>	<code>Route::post('/produk', ...)</code>

Routing dengan Closure: Menangani Respon Instan

Fungsi Anonim (Closure)

- **Respon Cepat:** Closure digunakan untuk rute sederhana yang tidak memerlukan logika bisnis kompleks.
- **Pengembangan Prototipe:** Sangat berguna untuk proses testing, mocking, dan merancang prototipe rute dengan instan.
- **Kembalian Variatif:** Bisa mengembalikan teks murni, JSON array, atau memanggil render berkas Blade views.

```
// Penerapan Closure di web.php

<?php
use Illuminate\Support\Facades\Route;

// 1. Mengembalikan string langsung
Route::get('/hallo', function () {
    return 'Selamat Datang di Pemrograman Framework!';
});

// 2. Mengembalikan array (otomatis diconvert ke JSON)
Route::get('/api/status', function () {
    return ['status' => 'aktif', 'code' => 200];
});

// 3. Memanggil file resources/views/welcome.blade.php
Route::get('/', function () {
    return view('welcome');
});
```

BAGIAN 03

PARAMETER ROUTING DINAMIS

Parameter Wajib (Required): Mengambil Data dari URL

Definisi Parameter Wajib

- **Tanda Kurung Kurawal:** Dideklarasikan di dalam rute menggunakan kurung kurawal `{nama\_parameter}`.
- **Ketergantungan Data:** Rute tidak akan bisa diakses jika parameter tersebut dilewatkan (error 404).
- **Urutan Argumen:** Diteruskan ke argumen fungsi closure secara berurutan dari kiri ke kanan.

```
// Implementasi Parameter Wajib

// Contoh 1: Satu parameter wajib
Route::get('/mahasiswa/{nrp}', function ($nrp) {
    return "Profil Mahasiswa dengan NRP: " . $nrp;
});
// Akses: /mahasiswa/5025201000

// Contoh 2: Multi-parameter wajib
Route::get('/blog/{id}/{slug}', function ($id, $slug) {
    return "Membaca artikel #{$id} dengan slug: {$slug}";
});
// Akses: /blog/42/laravel-routing-dasar

// Nama argumen bebas, tapi disarankan sama agar tidak membingungkan
Route::get('/user/{id}', function ($userId) {
    return 'User ID: ' . $userId;
});
```

Parameter Opsional (Optional): Fleksibilitas Navigasi

Definisi Parameter Opsional

- **Tanda Tanya (?):** Ditandai dengan simbol tanda tanya `{parameter?}` di akhir nama parameter rute.
- **Nilai Default Argumen:** Wajib memberikan nilai default (biasanya `null`) pada argumen penampung di closure.
- **Menolak Error 404:** Halaman tetap terbuka secara normal meskipun parameter di URL dikosongkan.

// Implementasi Parameter Opsional

// Contoh 1: Parameter opsional sederhana

```
Route::get('/dosen/{nama?}', function ($nama = null) {  
    if ($nama) {  
        return "Menampilkan detail Dosen: " . $nama;  
    }  
    return "Menampilkan daftar seluruh Dosen";  
});
```

// Akses 1: /dosen/ahmad (Menampilkan detail)

// Akses 2: /dosen (Menampilkan daftar seluruh)

// Contoh 2: Parameter opsional dengan penanganan default manual

```
Route::get('/laporan/{bulan?}', function ($bulan = null) {  
    $targetBulan = $bulan ?? date('m');  
    return "Laporan untuk bulan: " . $targetBulan;  
});
```

Menetapkan Nilai Default pada Parameter Opsional

Konsep Default Value

- **Inisialisasi Eksplisit:** Mendefinisikan string atau angka default langsung pada variabel argumen (misal: `\$status = 'aktif'`).
- **Fallback Cepat:** Menyediakan data cadangan instan untuk query tanpa perlu menuliskan percabangan if-else tambahan.
- **Keterbacaan Kode:** Memangkas baris penanganan nilai null dan merapikan visual fungsi.

// Penerapan Nilai Default

// Contoh 1: Default String

```
Route::get('/arsip/{kategori?}', function ($kategori = 'umum')  
{  
    return "Membuka folder arsip kategori: " . $kategori;  
});
```

// Akses 1: /arsip/akademik -> "kategori: akademik"

// Akses 2: /arsip -> "kategori: umum"

// Contoh 2: Default Nilai Angka

```
Route::get('/toko/produk/{page?}', function ($page = 1) {  
    return "Menampilkan produk halaman ke-" . $page;  
});
```

// Akses 1: /toko/produk/5 -> "halaman ke-5"

// Akses 2: /toko/produk -> "halaman ke-1"

Aturan Emas Penulisan Parameter Routing

Aturan Nama Parameter

- **Karakter Alfabetis:** Hanya boleh menggunakan kombinasi huruf (a-z, A-Z), angka (0-9), dan garis bawah (_).
- **Dilarang Menggunakan Simbol:** Tidak diperbolehkan menyisipkan tanda hubung (-) atau simbol khusus lainnya.
- **Sensitivitas Huruf:** Sebaiknya gunakan camelCase atau snake_case secara konsisten di seluruh tim.

Aturan Posisi Parameter

- **Opsional Harus di Akhir:** Parameter opsional `{param?}` wajib diletakkan di posisi paling kanan atau akhir rute.
- **Kelemahan Penempatan:** Jika opsional diletakkan sebelum parameter wajib, router akan bingung dan menghasilkan fatal syntax error.
- **Kesesuaian Urutan:** Laravel mencocokkan nilai URL ke variabel closure berdasarkan posisi urutan, bukan nama variabel.

Regular Expression: Mengamankan Parameter Rute

Fungsi Validasi Regex

- **Penyaringan Dini:** Membatasi format input yang boleh diterima rute sebelum dikirim ke database.
- **Pencegahan SQL Injection:** Memastikan parameter angka hanya berisi digit, mencegah penyusupan karakter aneh.
- **Fungsi Penaut:** Menggunakan method pembantu `where()` langsung setelah penulisan deklarasi rute.

```
// Penerapan Metode where()

// 1. Membatasi parameter hanya berupa ANGKA
Route::get('/user/{id}', function ($id) {
    return 'User ID: ' . $id;
})->where('id', '[0-9]+');
// Cocok: /user/123
// Error 404: /user/ahmad

// 2. Membatasi parameter hanya berupa HURUF
Route::get('/kategori/{nama}', function ($nama) {
    return 'Kategori: ' . $nama;
})->where('nama', '[A-Za-z]+');

// 3. Validasi multi-parameter sekaligus
Route::get('/mahasiswa/{nrp}/{nama}', function ($nrp, $nama) {
    return "NRP: {$nrp} - Nama: {$nama}";
})->where([
    'nrp' => '[0-9]+',
    'nama' => '[A-Za-z\s]+'
]);
```

BAGIAN 04

FITUR ROUTING LANJUTAN

Named Routes: Solusi Cerdas Mengelola Tautan

Konsep Named Routes

- **Pemberian Nama Unik:** Mengaitkan nama tertentu pada rute dengan menambahkan berantai method `->name()`.
- **Solusi Anti-Hardcode:** Menghindari penulisan URL manual secara berulang di controller maupun file Blade.
- **Kemudahan Pemeliharaan:** Bila URL rute diubah (misal `/tentang-kami` menjadi `/tentang`), tautan tidak akan patah.

```
// Mendefinisikan & Memanggil Rute Bernama

// 1. Definisi Rute Bernama di routes/web.php
Route::get('/akademik/profil-dosen', function () {
    return 'Profil Lengkap Dosen ITS';
})->name('dosen.profil');

// 2. Memanggil via Controller / Redirect
Route::get('/redirect-dosen', function () {
    return redirect()->route('dosen.profil');
});

// 3. Memanggil di dalam file Blade (HTML Template)
// <a href="{{ route('dosen.profil') }}">Detail Dosen</a>

// 4. Rute bernama dengan parameter dinamis
Route::get('/detail/{id}', function ($id) {
    return 'Detail: ' . $id;
})->name('produk.detail');

// Cara panggil: route('produk.detail', ['id' => 5])
```

Route Groups: Mengelompokkan Rute demi Kerapian

Keuntungan Route Grouping

- **Menolak Duplikasi:** Mengelompokkan rute yang berbagi konfigurasi yang sama agar baris kode tetap ramping.
- **Prefix URL:** Menambahkan string awalan otomatis pada URL (misal menambahkan `/admin/` di depan rute).
- **Penumpukan Keamanan:** Menerapkan pembatasan hak akses (middleware) secara masal untuk sekelompok rute.

```
// Penerapan Route::prefix()

// Mengelompokkan rute admin dengan prefix
Route::prefix('admin')->group(function () {

    // Diakses via URL: /admin/dashboard
    Route::get('/dashboard', function () {
        return 'Dashboard Admin';
    });

    // Diakses via URL: /admin/laporan
    Route::get('/laporan', function () {
        return 'Laporan Keuangan Admin';
    });

    // Diakses via URL: /admin/users
    Route::get('/users', function () {
        return 'Kelola Data Pengguna';
    });

});
```

Route::resource(): Otomatisasi Pola CRUD Modern

Sekilas pandang model routing resource yang langsung mendaftarkan 7 rute standar CRUD industri.

HTTP Verb	URL Path	Controller Method	Fungsi / Deskripsi
GET	/mahasiswa	index	Menampilkan tabel daftar seluruh mahasiswa.
GET	/mahasiswa/create	create	Menampilkan formulir tambah mahasiswa baru.
POST	/mahasiswa	store	Memproses kiriman formulir & menyimpan ke DB.
GET	/mahasiswa/{id}	show	Menampilkan detail satu data mahasiswa.
GET	/mahasiswa/{id}/edit	edit	Menampilkan formulir edit data mahasiswa.
PUT / PATCH	/mahasiswa/{id}	update	Memproses kiriman data edit untuk diperbarui di DB.
DELETE	/mahasiswa/{id}	destroy	Menghapus permanen data mahasiswa dari database.

Menginspeksi Rute dengan php artisan route:list

Pentingnya route:list

- **Audit Navigasi:** Menampilkan seluruh daftar rute aplikasi lengkap dengan HTTP verb, URI, Name, dan Controller Action.
- **Debugging Cepat:** Alat bantu utama jika rute dirasa tidak cocok, bentrok, atau tidak dapat diakses.
- **Filtering Handal:** Menyediakan flag khusus untuk memfilter tampilan rute berdasarkan pola tertentu.

// Command Terminal untuk Inspeksi

Menampilkan semua rute terdaftar di aplikasi

```
php artisan route:list
```

Menampilkan rute dengan filter kolom tertentu

```
php artisan route:list --columns=method,uri,name
```

Melakukan filter berdasarkan awalan URI

```
php artisan route:list --path=admin
```

Menyembunyikan rute-rute bawaan framework vendor

```
php artisan route:list --except-vendor
```

Mengurutkan berdasarkan URI secara ascending

```
php artisan route:list --sort=uri
```

Menepis Ancaman Web via CSRF Protection

Apa itu CSRF?

- **Definisi Ancaman:** Cross-Site Request Forgery (CSRF) adalah taktik eksploitasi di mana situs jahat memaksa browser mengirim request tanpa izin.
- **Aksi Berbahaya:** Dapat melakukan aksi penghapusan, transfer dana, atau pembaruan password secara gaib.
- **Target Rentan:** Semua HTTP request yang mengubah state di server (POST, PUT, DELETE) wajib dilindungi.

Penanganan Laravel secara Native

- **Verifikasi Token:** Laravel otomatis melacak token sesi rahasia untuk memvalidasi keaslian pembuat request.
- **Directive @csrf:** Wajib menambahkan deklarasi `@csrf` di setiap form HTML agar tidak terhadang error 419 Page Expired.
- **Pengecualian Rute:** Dapat disesuaikan di file konfigurasi jika rute web tertentu tidak membutuhkan token (misal: webhook).

Refleksi Bersama: Routing Terbaik Skala Industri

Mengapa Named Route Dianggap Sebagai Standar Wajib?

Bayangkan aplikasi e-commerce besar memiliki ratusan file template. Jika link ke halaman '/checkout' ditulis secara manual (hardcoded) dan di kemudian hari manajer meminta perubahan URL menjadi '/bayar-sekarang', Anda harus mencari dan mengedit ratusan file tersebut secara manual. Dengan named route, Anda cukup mengubahnya di satu file web.php saja!

Kapan Memilih Parameter Opsional dibanding Route Terpisah?

Gunakan parameter opsional `{param?}` jika halaman yang ditampilkan memiliki struktur antarmuka dan komponen yang mirip, hanya berbeda pada filter data (misal: /produk vs /produk/laptop). Buat route terpisah jika logika proses, Controller yang menangani, dan struktur view-nya benar-benar berbeda total.

BAGIAN 05

LATIHAN CONTOH SOAL & SOLUSI KODINGAN

Soal 1: Route GET Dasar dengan Parameter Wajib

Skenario Permasalahan

- **Tugas Mahasiswa:** Buatlah sebuah rute GET dengan alamat `/kelas/{kode}` di dalam berkas web.php.
- **Syarat Output:** Rute tersebut harus menerima parameter kode kelas (misal: 'PBKK-A') dan mengembalikan string teks respon.
- **Format Respon:** Output yang diharapkan adalah: "Membuka ruang kelas mata kuliah: [kode_kelas]"

// Jawaban Solusi Kodingan

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
// Solusi Deklarasi Route
```

```
Route::get('/kelas/{kode}', function ($kode) {
```

```
    // Membersihkan input secara opsional untuk keamanan
```

```
    $cleanKode = htmlspecialchars($kode);
```

```
    return "Membuka ruang kelas mata kuliah: " . $cleanKode;
```

```
});
```

```
// Contoh Skenario Uji Coba di Browser:
```

```
// URL: http://127.0.0.1:8000/kelas/PBKK-B
```

```
// Output: "Membuka ruang kelas mata kuliah: PBKK-B"
```

Soal 2: Menangani Parameter Opsional & Default

Skenario Permasalahan

- **Tugas Mahasiswa:** Buat rute `/laporan/{tahun?}` di mana parameter tahun bersifat opsional.
- **Syarat Logika:** Bila tahun disertakan, tampilkan teks sesuai tahun tersebut. Bila kosong, gunakan nilai fallback default '2026'.
- **Format Respon:** Mengembalikan respon bertuliskan: "Menyusun Laporan Tahunan Periode: [tahun]"

// Jawaban Solusi Kodingan

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
// Solusi Deklarasi Route
```

```
Route::get('/laporan/{tahun?}', function ($tahun = '2026') {
```

```
    // Memastikan input hanya berupa karakter aman
```

```
    $cleanTahun = htmlspecialchars($tahun);
```

```
    return "Menyusun Laporan Tahunan Periode: " . $cleanTahun;
```

```
});
```

```
// Skenario Uji 1 (Tanpa Parameter):
```

```
// URL: http://127.0.0.1:8000/laporan
```

```
// Output: "Menyusun Laporan Tahunan Periode: 2026"
```

```
// Skenario Uji 2 (Dengan Parameter):
```

```
// URL: http://127.0.0.1:8000/laporan/2028
```

```
// Output: "Menyusun Laporan Tahunan Periode: 2028"
```

Soal 3: Pengelompokan Rute Ber-prefix Khusus

Skenario Permasalahan

- **Tugas Mahasiswa:** Kelompokkan rute panel dosen di bawah awalan URI '/dosen'.
- **Detail Rute:** Buat dua rute di dalam grup tersebut: '/dosen/jadwal' dan '/dosen/nilai'.
- **Format Respon:** Masing-masing rute harus mengembalikan string informatif yang sesuai dengan tujuannya.

// Jawaban Solusi Kodingan

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
// Solusi Grouping dengan Prefix
```

```
Route::prefix('dosen')->group(function () {
```

```
    // Rute 1: Diakses via /dosen/jadwal
```

```
    Route::get('/jadwal', function () {
```

```
        return "Menampilkan Jadwal Mengajar Dosen Semester
```

```
Ini";
```

```
    });
```

```
    // Rute 2: Diakses via /dosen/nilai
```

```
    Route::get('/nilai', function () {
```

```
        return "Halaman Input Nilai Mahasiswa Terintegrasi";
```

```
    });
```

```
});
```

Soal 4: Named Route & Alur Redirect Internal

Skenario Permasalahan

- **Tugas Mahasiswa:** Definisikan sebuah rute bernama '/its-center' dan berikan nama rute 'its.pusat'.
- **Rute Pemicu:** Buatlah rute pemicu '/pindah' yang bertugas me-redirect pengguna ke rute 'its.pusat' secara otomatis.
- **Format Respon:** Rute '/its-center' mengembalikan teks: "Selamat! Anda telah sampai di ITS Pusat"

// Jawaban Solusi Kodingan

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
// 1. Rute tujuan yang diberi nama unik
Route::get('/its-center', function () {
    return "Selamat! Anda telah sampai di ITS Pusat";
})->name('its.pusat');
```

```
// 2. Rute pemicu redirect
Route::get('/pindah', function () {
    // Redirect aman menggunakan referensi nama rute
    return redirect()->route('its.pusat');
});
```

```
// Skenario Jalannya Request:
```

```
// Pengguna mengakses URL /pindah -> Otomatis dialihkan ke /its-center
```

Soal 5: Aplikasi Kalkulator URL dengan 3 Parameter

Skenario Permasalahan

- **Tugas Mahasiswa:** Rancang rute kalkulator dinamis `/hitung/{a}/{op}/{b}` di mana parameter `{op}` adalah operasi matematika.
- **Logika Operasi:** Gunakan switch-case untuk menangani operasi tambah, kurang, kali, dan bagi.
- **Penanganan Kasus Khusus:** Wajib mencegah eror pembagian dengan angka nol dan tampilkan pesan khusus jika terdeteksi.

// Jawaban Solusi Kodingan

```
Route::get('/hitung/{a}/{op}/{b}', function ($a, $op, $b) {  
    // Validasi tipe data numerik  
    if (!is_numeric($a) || !is_numeric($b)) {  
        return 'Error: Parameter input wajib berupa angka!';  
    }  
    switch ($op) {  
        case 'tambah': $hasil = $a + $b; break;  
        case 'kurang': $hasil = $a - $b; break;  
        case 'kali':    $hasil = $a * $b; break;  
        case 'bagi':  
            if ($b == 0) return 'Error: Pembagian dengan nol terdeteksi!';  
            $hasil = $a / $b; break;  
        default: return 'Error: Operasi tidak didukung!';  
    }  
    return "Hasil Kalkulasi {$a} {$op} {$b} = " . $hasil;  
})->where(['a' => '[0-9]+', 'b' => '[0-9]+']);
```

BAGIAN 06

TUGAS MANDIRI & TANTANGAN MINGGUAN

Tugas Mandiri: Laravel Local Sandbox (Langkah 1/5)

Deskripsi dan Konseptual Tugas Mandiri 1 Minggu

Misi Utama Tugas:



Setiap mahasiswa wajib membangun sebuah aplikasi profile akademis statis mandiri menggunakan local routing sandbox Laravel.

Studi Kasus Profil:



Menyajikan data diri, proyeksi ide akhir semester platform Agentic AI, dan riwayat studi menggunakan arsitektur pemetaan rute terstruktur.

Tujuan Kompetensi:



Memastikan mahasiswa lancar melakukan inisialisasi framework, memahami fungsi routing web, serta mampu menerapkan named routes dan pengelompokan rute prefix.

Spesifikasi Teknis Tugas (Langkah 2/5)

Aturan Kaku yang Wajib Dipenuhi dalam Penulisan Kode



Rute 1: Halaman Home (/)

Menampilkan sambutan selamat datang hangat khas ITS dan profil singkat mahasiswa yang interaktif.



Rute 2: Detail Profil (/mahasiswa/{nrp})

Menggunakan parameter wajib NRP. Rute harus mengembalikan informasi profil lengkap pemilik NRP tersebut.



Rute 3: Ide Platform Agentic AI (/agent/{tema?})

Menerapkan parameter opsional tema platform AI. Bila tema diisi, tampilkan penjelasannya. Bila kosong, gunakan nilai fallback default: 'General Assistant Agent'.



Penerapan Named Rute:

Seluruh rute WAJIB dideklarasikan menggunakan metode `->name()` untuk mempermudah navigasi internal.

Tantangan Tambahan (Langkah 3/5) — Challenge Nilai A+

Fitur Opsional untuk Mengasah Kemampuan Tingkat Lanjut

Tantangan 1: Pengamanan Parameter (Regex)



Terapkan Regular Expression `where()` pada rute `/mahasiswa/{nrp}` sehingga hanya mau menerima kombinasi angka bulat sepanjang 10 digit (NRP ITS standar).

Tantangan 2: Kalkulator Portofolio Akademis



Buat rute kalkulator IPK otomatis: `/hitung-ipk/{ip1}/{ip2}` untuk menjumlahkan dan mencari rata-rata IP mahasiswa dalam dua semester.

Tantangan 3: Grouping & Fallback Route



Kelompokkan rute profil akademis di bawah prefix `/dashboard`. Serta buat rute penanganan halaman tidak ditemukan (`Route::fallback()`) yang rapi.

Rubrik Penilaian Tugas Mandiri (Langkah 4/5)

Skema evaluasi penilaian transparan yang akan diterapkan dalam menilai hasil tugas mandiri mahasiswa.

Kriteria Penilaian	Bobot	Indikator Keberhasilan Utama
Kesesuaian Spesifikasi Teknis	40%	Seluruh rute yang diminta (3 rute wajib) berjalan mulus tanpa error, parameter required/optional terpasang benar.
Kerapian Kode & Standar PSR	20%	Struktur penulisan kode PHP bersih, mengikuti indentasi standar, penamaan variabel jelas, dan komentar memadai.
Named Routes & Pemeliharaan	20%	Implementasi named routes secara penuh di seluruh rute serta tidak adanya hardcoded URL di dalam respon.
Keberhasilan Challenge (A+)	10%	Sukses menerapkan regex pengaman, kalkulator IPK, dan rute fallback dengan logika yang kokoh.
Kemampuan Demo & Pemahaman	10%	Kejelasan dalam mempresentasikan jalannya alur kode dan ketepatan menjawab pertanyaan acak dosen saat demo.

Ketentuan Pengumpulan & Sesi Demo (Langkah 5/5)

Prosedur Akhir Pengumpulan Tugas Mandiri

Unggah ke GitHub Personal:



Buat repositori publik di akun GitHub Anda masing-masing. Pastikan file `.env` dan folder `vendor/` tidak ikut terunggah (gunakan `.gitignore` bawaan).

Batas Waktu Pengumpulan LMS:



Kumpulkan tautan (link) repositori GitHub Anda ke portal LMS kampus paling lambat H-1 sebelum Pertemuan 3 dimulai.

Sesi Live Demo Kode Acak:



Di awal Pertemuan 3, Dosen akan memilih 5 mahasiswa secara acak untuk melakukan demo aplikasi dan menjelaskan struktur rutanya selama maksimal 5 menit.

SELAMAT MENGEKSPLORASI LARAVEL!

Routing adalah pintu gerbang awal dari setiap aplikasi web yang kokoh.
Sampai jumpa di Pertemuan 3 dengan topik Request Lifecycle, Form POST, dan CSRF Protection.

Departemen Teknik Informatika - FTEIC - Institut Teknologi Sepuluh Nopember